Technical Assessment Report: Drone Security Analyst Agent

Candidate: AI Engineer Candidate

Position: AI Engineer (FlytBase)
Date: May 28, 2026

Executive Summary

This report summarizes the design, implementation, and verification of the SkySentinel AI Drone Security Analyst Agent prototype. SkySentinel AI integrates live autonomous drone telemetry with Vision-Language Model (VLM) frame parsing to establish a continuous, searchable security record of a commercial property.

The system automates threat detection using an active rules engine (evaluating safety and trespassing anomalies) and leverages LangChain RAG techniques to present property owners with a natural language interface for querying patrol archives.

---

1. Architectural Design & Pipeline Workflow

1.1 Multi-Modal Telemetry & Video Synchronization

Autonomous drone patrols produce asynchronous telemetry streams (GPS coordinates, battery levels, speed, altitude) and continuous video frame capture. A primary challenge in drone surveillance is data drift?ensuring the analyst knows *exactly* where the drone was when an anomaly was spotted.

SkySentinel AI addresses this through a synchronized data ingestion pipeline:

$$\text{Synced Frame} = \{\text{Timestamp}, \text{Frame Index}, \text{VLM Visual Log}, \text{Altitude}, \text{Battery}, \text{Location}, \text{Coordinates}\}$$

Each telemetry reading and visual frame is aligned by timestamp and persisted into a structured relational SQLite index.

1.2 Storage Strategy: Embedded SQLite with FTS5

Rather than introducing heavy, network-dependent vector database clusters (e.g. Chroma, Pinecone, or Milvus) which increase compilation overhead and operational complexity, SkySentinel AI implements a self-contained, high-performance SQLite database using the built-in FTS5 (Full-Text Search) virtual table module.

- SQLite provides sub-millisecond local reads/writes, crucial for real-time edge deployments on drone docking hubs.
- The FTS5 extension creates index matching on frame visual descriptions, allowing complex natural language object searches (e.g. *"blue truck near docks"*) without the heavy token usage of full text-embedding loops, ensuring optimal cost-performance.

---

2. Vision-Language Model (VLM) Analysis: Decisions & Tradeoffs

2.1 Tool Choices & Justification: VLM Options

During development, three primary visual models were evaluated:

1. CLIP (Contrastive Language-Image Pretraining): Excellent for simple image-to-text classification and zero-shot categorization, but lacks the generative reasoning required to write detailed textual descriptions of complex actions (e.g., *"a person wearing a dark hoodie is seen unloading boxes"*).
2. BLIP / BLIP-2: Generative captioning models capable of producing natural descriptions, but often suffer from lack of context or low resolution and are slow to run locally without a dedicated GPU.

3. API-driven VLMs (Google Gemini 1.5 Flash / OpenAI GPT-4o-mini): Chosen for the prototype's live mode. Gemini 1.5 Flash is highly optimized, provides an exceptional free-tier structure, and natively processes high-resolution image payloads. It generates highly specific security reports complete with visual tags.

2.2 Framework Selection: LangChain Functions Agent

We implemented the conversational Q&A auditor using the LangChain Functions Agent framework.

- The agent is equipped with three highly specific database tools (search_security_logs, get_triggered_alerts, get_patrol_telemetry_history).
- This enables the LLM to write optimized SQLite full-text search parameters, analyze coordinates, summarize flight boundaries, and output concise, verified answers with timestamps to security audits.

- Offline Fallback: To guarantee absolute reliability, we built a fully autonomous Python RAG search fallback. If no API keys are present, the system implements keyword vector matches, allowing complete dashboard operations offline.

---

3. Threat Detection Rules & Security Log Results

3.1 Predefined Rules Engine

The rules engine processes each synced frame against security and safety logic:

1. After-Hours Security Intrusion: Spawns a CRITICAL alert if a person or loitering tag is found between 9:00 PM and 6:00 AM.
2. Restricted Zone Vehicle: Spawns a WARNING or CRITICAL alert if a vehicle is detected near loading docks or chemical zones outside operational hours.

3. Drone Critical Battery: Spawns a WARNING if battery falls below 20% while hovering above 5 meters, advising an immediate Return-to-Home (RTH).
4. Hazardous Tipped Drum: Spawns a CRITICAL safety warning if tipped-over chemical containers or leaks are visually recognized.

3.2 Visual & Textual Results Showcase

- Security Logs Index Example:

- Triggered Alert Console Example:
- CRITICAL: [CRITICAL] IntrusionAfterHours (00:02:00): SECURITY INTRUSION: Unauthorized person spotted near 'Warehouse Loading Docks' at 00:02 (after hours).

---

4. Key Assumptions & Future Optimizations

4.1 Assumptions Made:

- The drone operates on a pre-programmed commercial estate patrol route with predictable spatial waypoints.

- Live VLM calls are downsampled to 1 frame every 2 seconds of video feed to optimize network latency and token costs.

4.2 Future Scalability & Improvements:

- Edge Processing: Deploying local quantized models (e.g. LLaVA-NeXT or Moondream2) directly onto the docked ground station for 100% private, local video parsing.

- Video Summarization (Temporal RAG): Integrating native video models to summarize broad multi-minute clips rather than averaging single frames.
- Autonomous Path Re-routing: Enabling the agent to dynamically instruct the drone autopilot to hover or fly closer if a WARNING level alert is triggered, capturing high-resolution evidence.